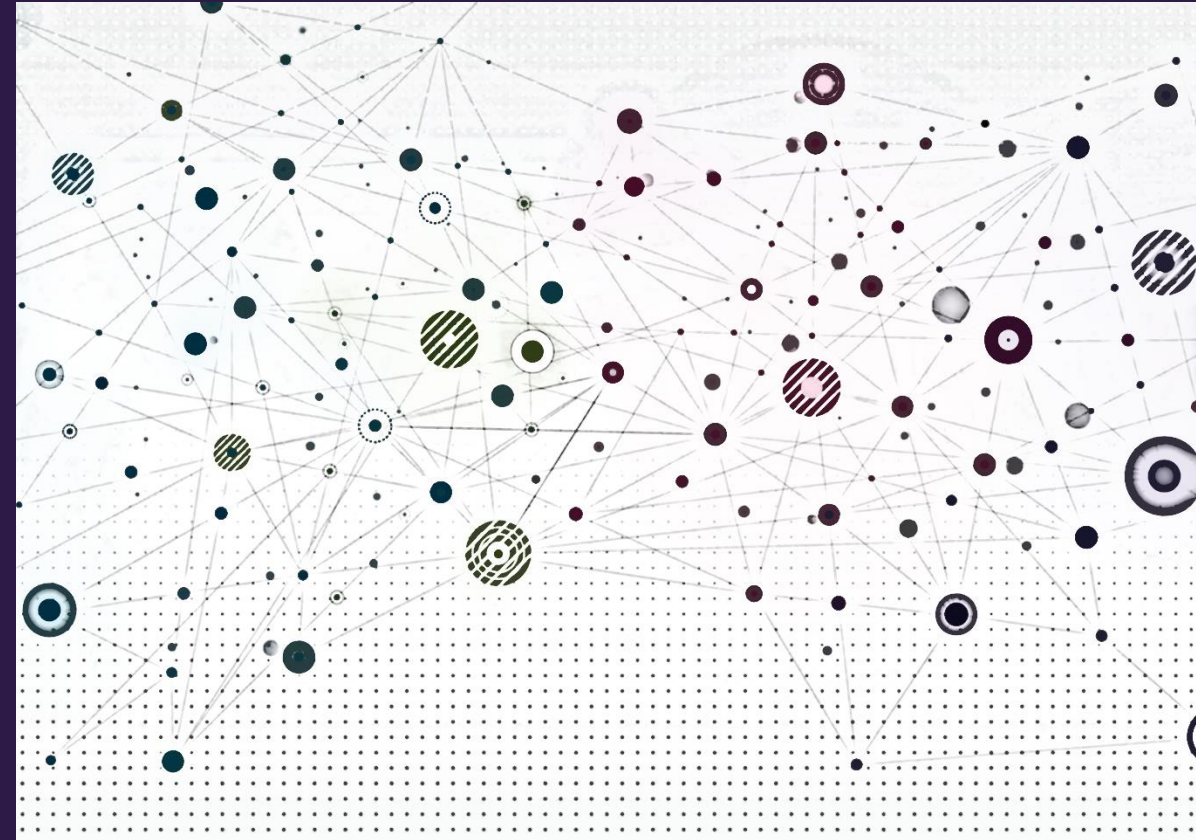


Generative graph models





Overview

- What is a generative graph model?
- Traditional graph generation methods
- Deep generative models
- Graphs as sequences
- GraphRNN
- Evaluation



What is a graph generative model?



What is a generative AI model?

Consider a **standard machine learning** classification model

$$f_{\theta}(x) = p(y)$$

- Given an input x (e.g., an image), output a label, y
- Formally, the model learns the probability distribution $p_{\theta}(y|x)$

Generative AI, is usually framed as creating the model

$$f_{\theta}(z) = p(x)$$

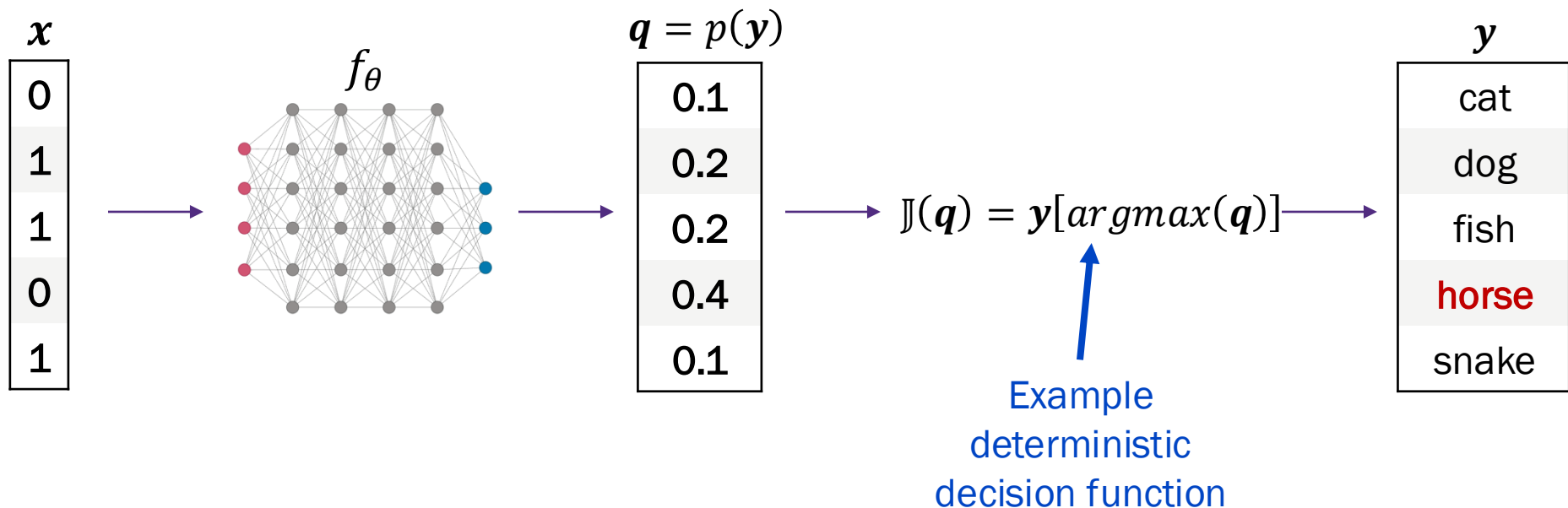
- Given an input z (e.g., label), output a sample (e.g., an image)
- Formally, the model learns the probability distribution $p_{\theta}(x|z)$

Are these paradigms fundamentally different?

What is a generative AI model?

Consider a **classification system**. It has two parts:

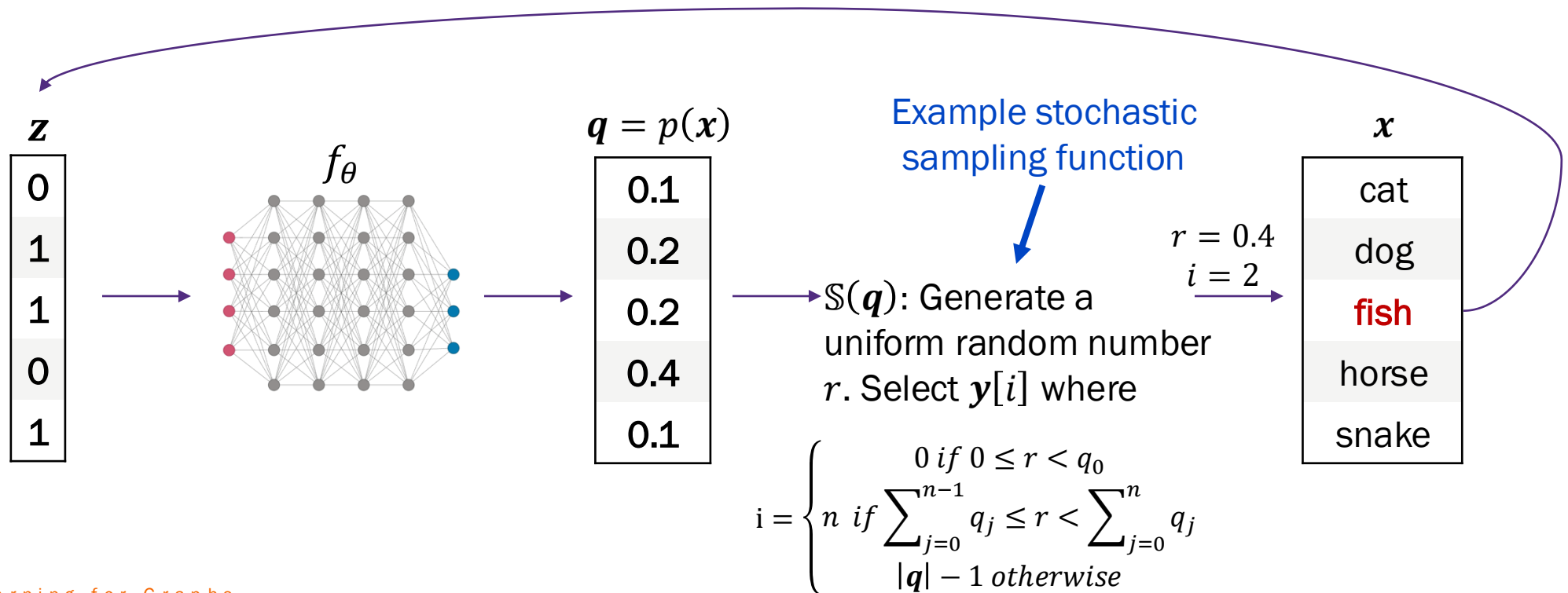
1. The model, $f_{\theta}(x)$, outputs the probability, $p(y)$, of a class label, y , for an input x
2. A **decision function**, $\mathbb{J}$, on $p(y)$ to **deterministically** selects the outcome label, $\hat{y}$ (e.g., if $p(y) > 0.5$, assign positive class)



What is a generative AI model?

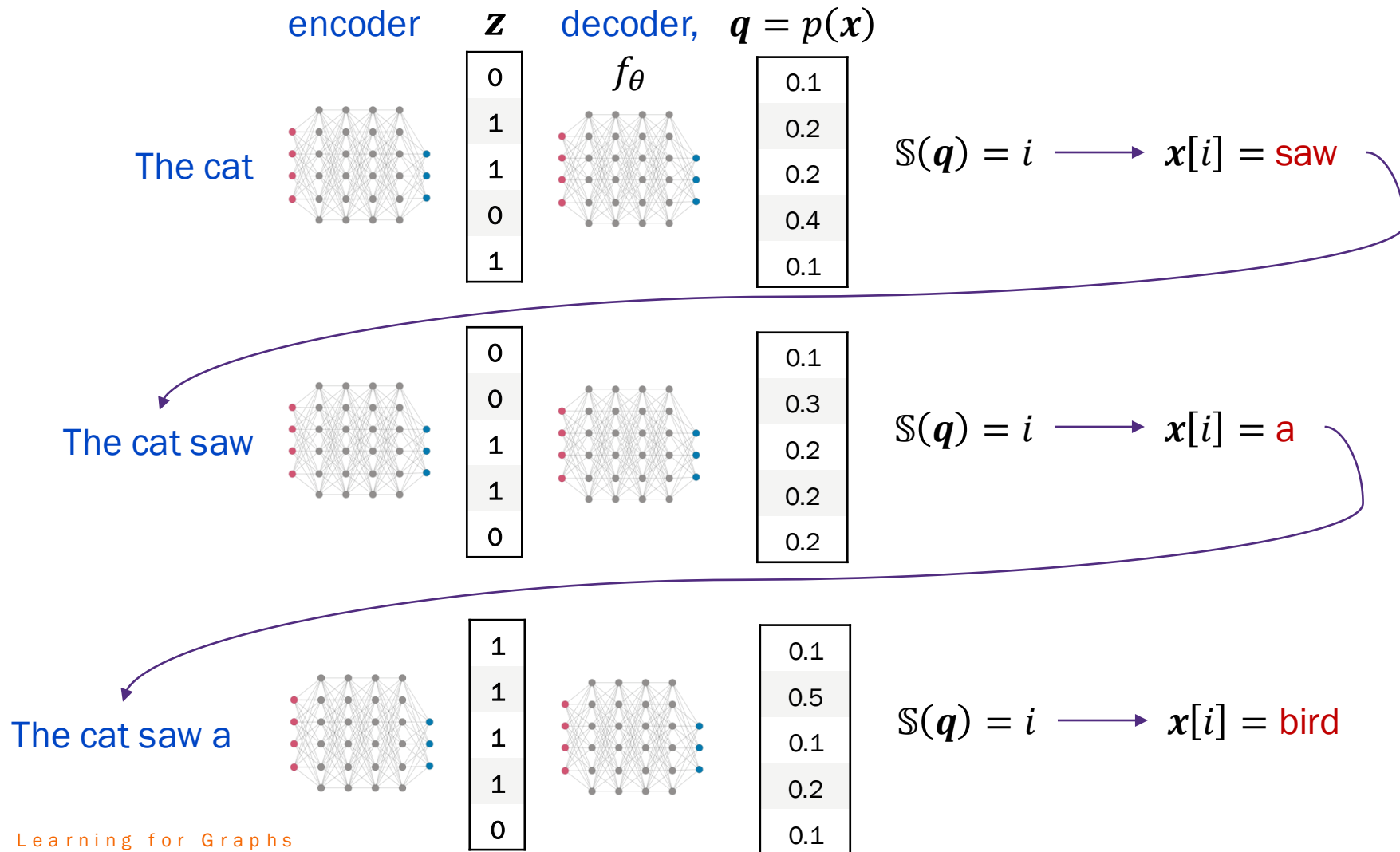
Consider a generative system. It has **two** parts:

1. The model, $f_\theta(z)$, outputs the probability, $p(x)$, of a sample, x , for an input z
2. A sampling function, S , on $p(x)$ **stochastically** selects the outcome sample, $\hat{x}$ (e.g., if $p(x) = 0.5$, flip a coin to decide)



What is a generative AI model?

Example: Generative language model



What is a generative AI model?

- Standard classification and generative AI both model a probability distribution $p_{\theta}(y|x)$ or $p_{\theta}(x|z)$ - the distinction in variable naming is unimportant
- The difference lies in the use of the probabilities to either deterministically decide a label vs. stochastically sampling
- There are substantial differences in model architectures and loss functions depending on the approach



Graph generation?

- So far, we've assumed the graph is given
- Models predict probability of node/edge/graph label or regression value
- Can we generate graphs instead?



Image credit: [vertabelo](#)

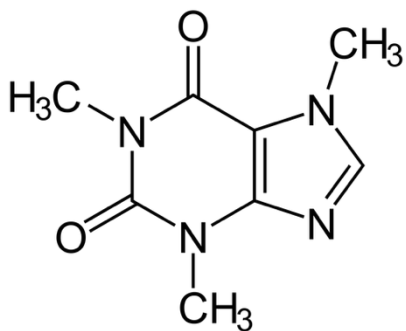


Image credit: [Chemistry StackExchange](#)

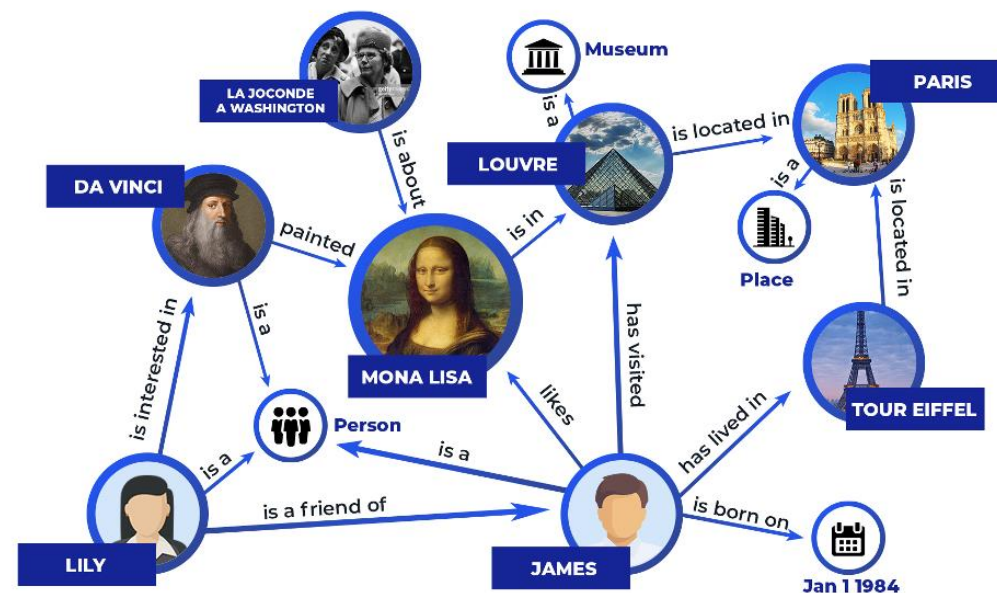
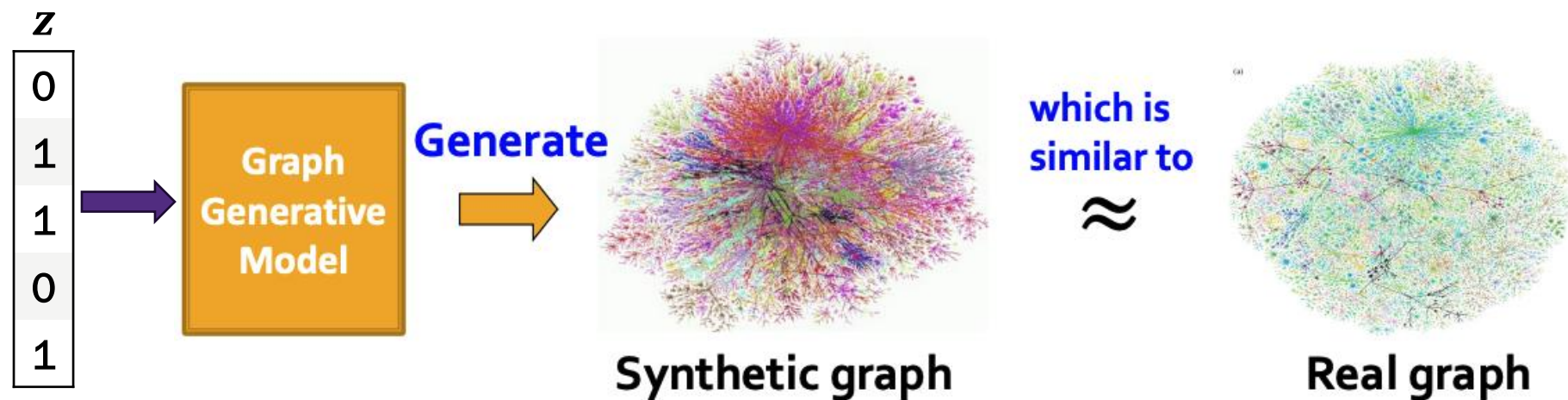


Image credit: [Zilliz](#)



Graph generation?

- Goal is to generate *realistic* graphs (i.e., graphs similar in distribution to graphs in our data)





Graph generation motivation

- Benchmarking graph methods (e.g., community detection)
- Epistemology of graph/network formulation
- Prediction of graph evolution (e.g., how will social networks evolve over time)
- Simulation of novel graphs (e.g., novel drugs)
- Anomaly detection (e.g., unusual user behavior)



Traditional graph generation methods





Traditional approach

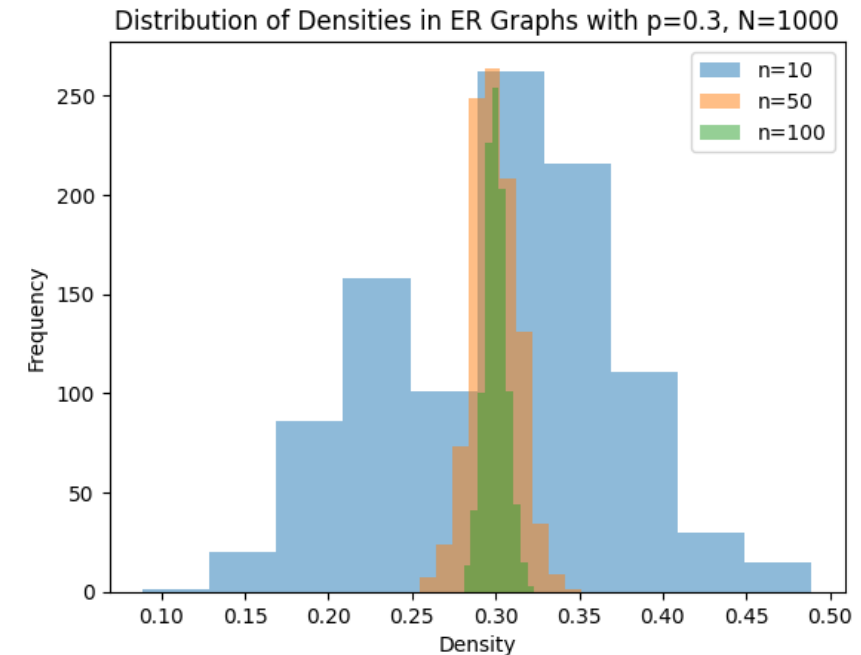
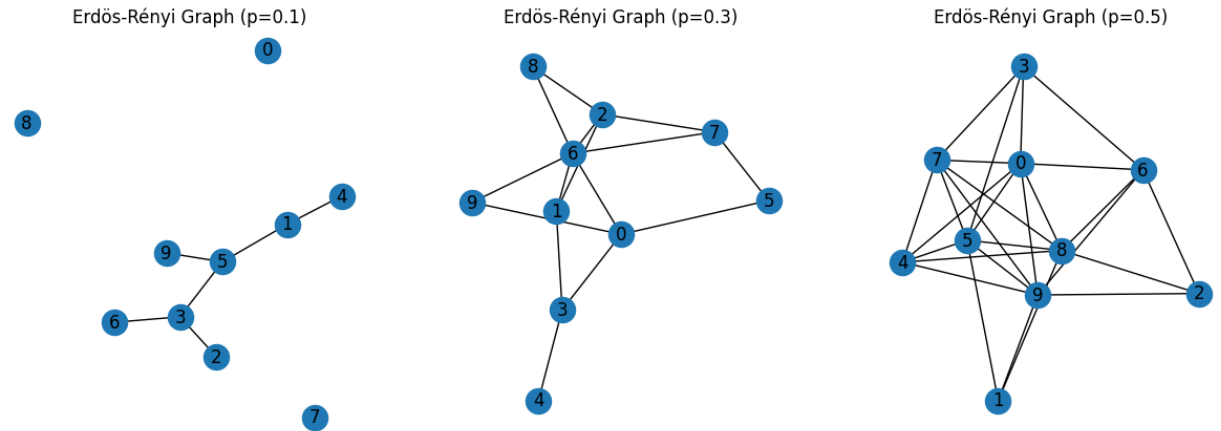
- Need to specify a generative process defining edge creation
- Usually framed as specifying the probability $p(\mathbf{A}[u, v] = 1)$ of an edge between nodes u and v
- Process needs to be tractable and lead to graphs with desired properties
- Analogous to feature engineering

Erdős-Rényi model

- Define the probability of an edge between any pair of nodes by $p(A[u, v] = 1) = r, \forall u, v \in \mathcal{V}, u \neq v$ where $r \in [0,1]$ is a parameter that controls graph density

$$\rho = \frac{m}{m_{max}} = \frac{2m}{n(n-1)} = \frac{c}{n-1}$$

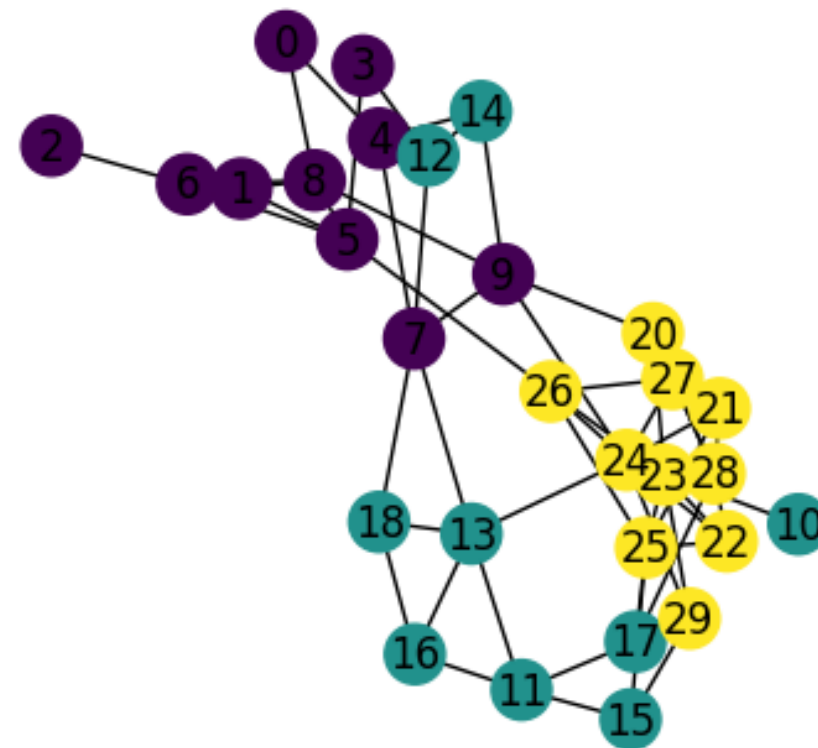
- Does not generate realistic graphs. For example, no control for degree distribution, community structure, clustering coefficients





Stochastic block models

- Specify the number of blocks, γ
- Assign probabilities $p_i(u \in b_i)$ of a node u belonging to block b_i

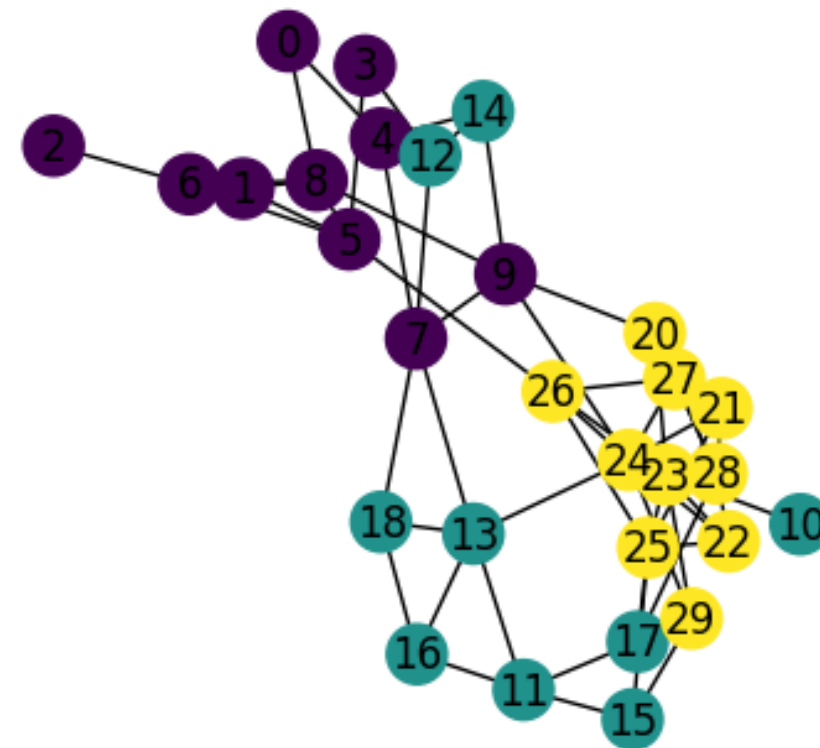


SBM generated graph
with three (3) blocks



Stochastic block models

- Specify the number of blocks, γ
- Assign probabilities $p_i(u \in b_i)$ of a node u belonging to block b_i
- Define the block probability matrix $\mathbf{B} \in \mathbb{R}^{\gamma \times \gamma}$ where $\mathbf{B}[i, j]$ is the probability of an edge occurring between a node in block b_i and a node in b_j

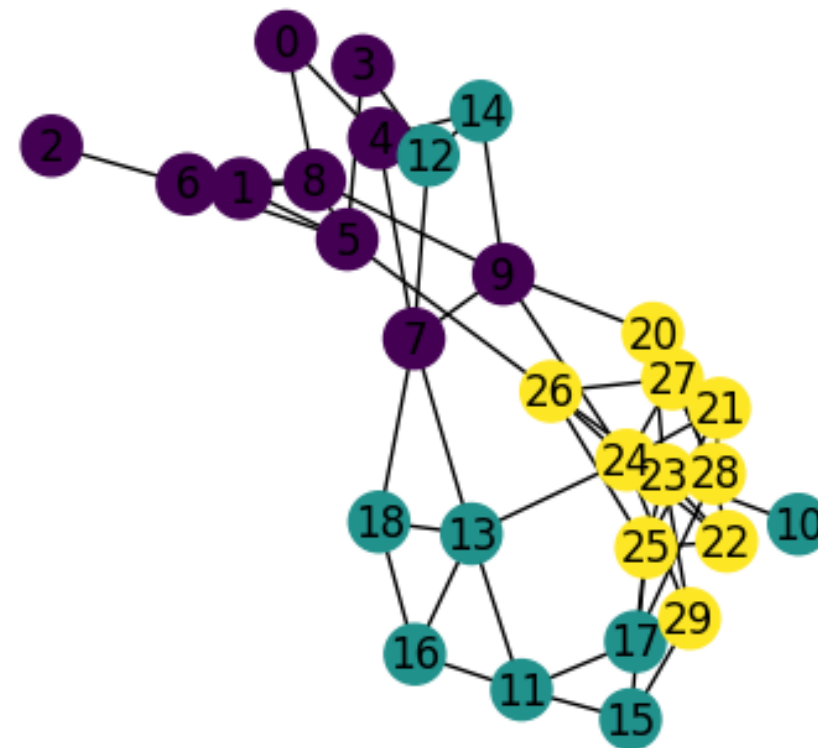


SBM generated graph
with three (3) blocks



Stochastic block models

- Specify the number of blocks, γ
- Assign probabilities $p_i(u \in b_i)$ of a node u belonging to block b_i
- Define the block probability matrix $\mathbf{B} \in \mathbb{R}^{\gamma \times \gamma}$ where $\mathbf{B}[i, j]$ is the probability of an edge occurring between a node in block b_i and a node in b_j
- The probability of an edge between nodes $u \in b_i$ and $v \in b_j$ is $p(\mathbf{A}[u, v] = 1) = \mathbf{B}[i, j]$

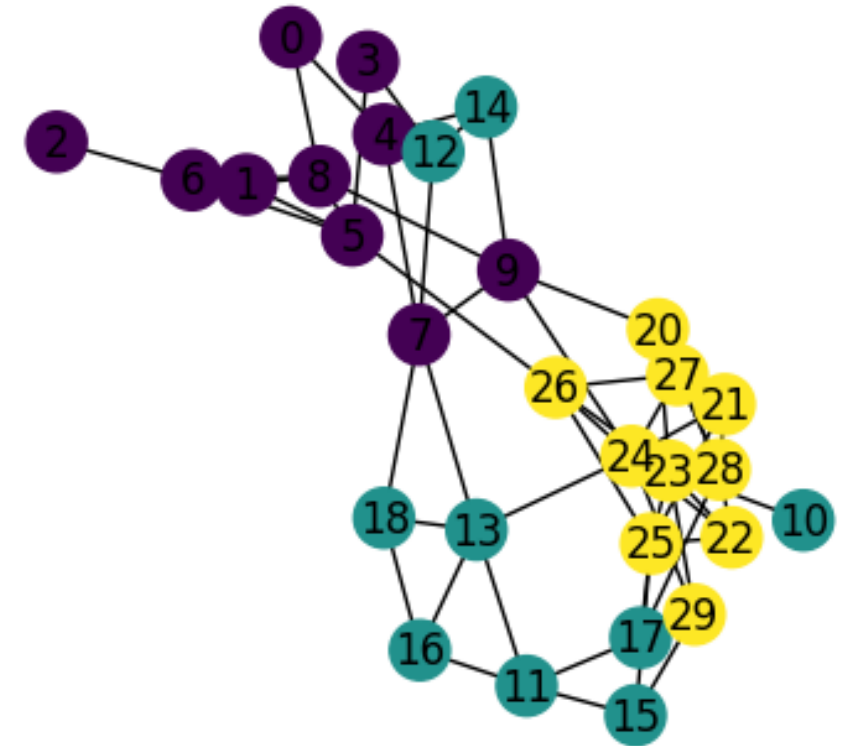


SBM generated graph
with three (3) blocks



Stochastic block models

- Specify the number of blocks, γ
- Assign probabilities $p_i(u \in b_i)$ of a node u belonging to block b_i
- Define the block probability matrix $\mathbf{B} \in \mathbb{R}^{\gamma \times \gamma}$ where $\mathbf{B}[i, j]$ is the probability of an edge occurring between a node in block b_i and a node in b_j
- The probability of an edge between nodes $u \in b_i$ and $v \in b_j$ is $p(\mathbf{A}[u, v] = 1) = \mathbf{B}[i, j]$
- Typically, $\mathbf{B}[i, i] = \alpha$ and $\mathbf{B}[i, j] = \beta \quad \forall i \neq j$
- Captures community structure but fails to capture structural characteristics of individual nodes (e.g., all nodes in a block have the same degree distribution)



SBM generated graph
with three (3) blocks

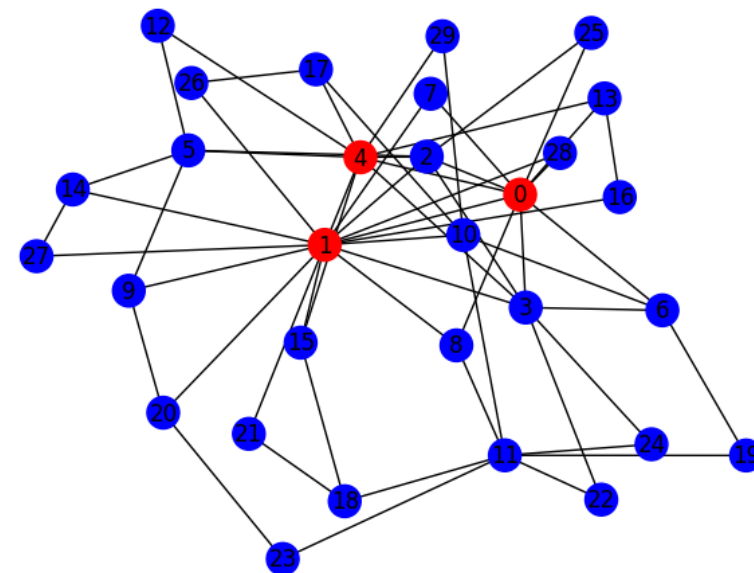


Preferential attachment

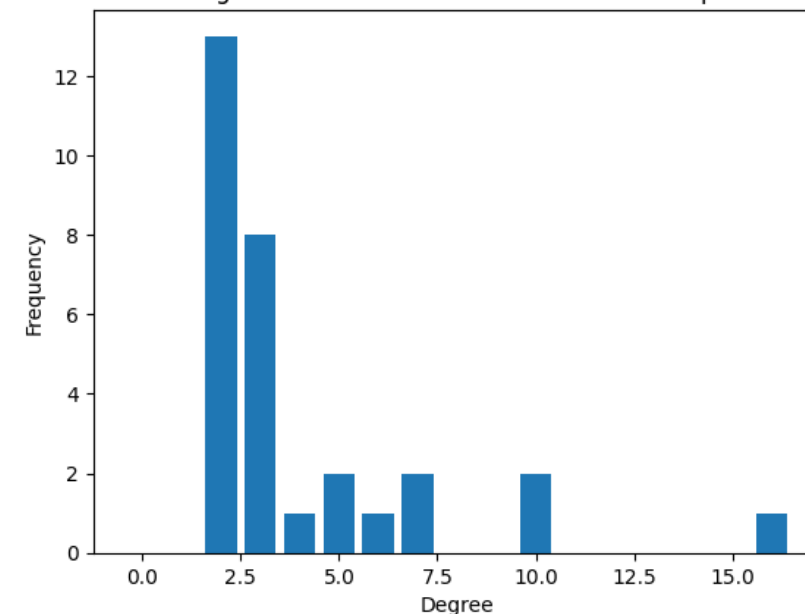
- Attempts to model node degree distributions observed in real-world graphs

Preferential attachment

- Attempts to model node degree distributions observed in real-world graphs
- Generates graphs that exhibit power-law (heavy tail) distributions where most nodes have low degree but some have very large degree



Degree Distribution of Pre-Attachment Graph





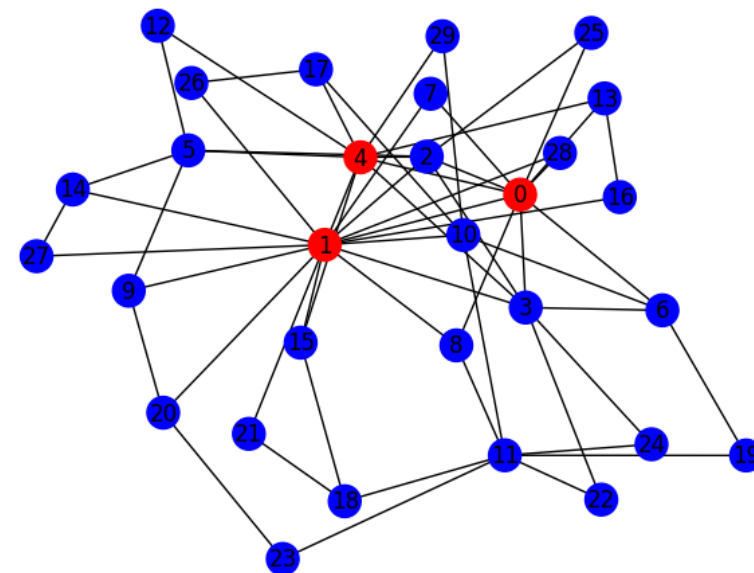
Preferential attachment

- Attempts to model node degree distributions observed in real-world graphs
- Generates graphs that exhibit power-law (heavy tail) distributions where most nodes have low degree but some have very large degree
- Approach
 - Initialize a fully connected graph with m_0 nodes. Choose integers, n and $m < m_0$
 - For $t \in [1, n - m_0]$, create a new node u , and connect it to m neighbors by sampling an existing node v with probability

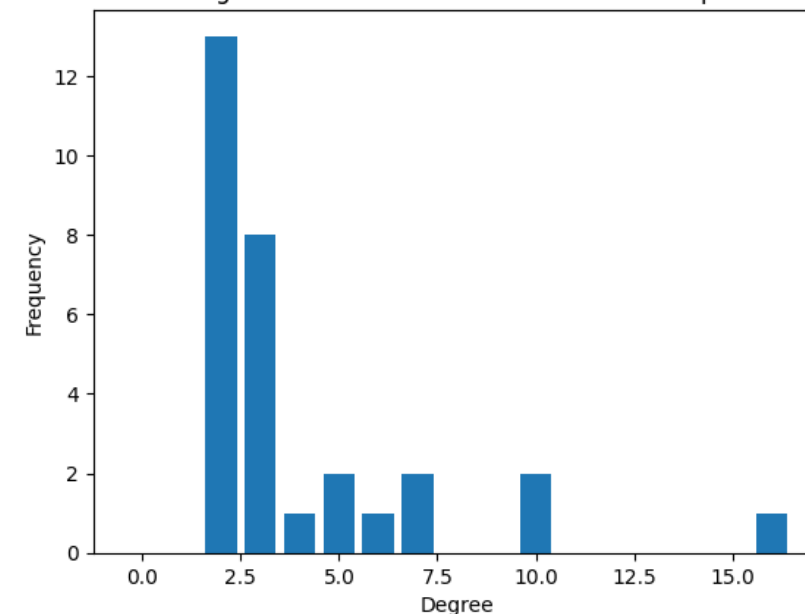
$$p(v) = \frac{d_v^{(t)}}{\sum_{v' \in \mathcal{V}^{(t)}} d_{v'}^{(t)}}$$

node degree at step t

nodes in graph at step t



Degree Distribution of Pre-Attachment Graph





Deep generative models





Graph generative model

Given observed graphs $\mathbb{G} = \{G_1, \dots, G_s\}$ sampled from a **data distribution** $p_{data}(G)$, learn a distribution model, $p_{model}(G)$ such that

$$p_{model}(G) \approx p_{data}(G)$$

Data distribution is generally unknown.
We only have observed samples





Graph generative model

Given observed graphs $\mathbb{G} = \{G_1, \dots, G_s\}$ sampled from a **data distribution** $p_{data}(G)$, learn a distribution model, $p_{model}(G)$ such that

$$p_{model}(G) \approx p_{data}(G)$$

Data distribution is generally unknown.
We only have observed samples

1. Assume $p_{model}(G)$ is parameterized on Θ
2. Apply some form of density estimation to ensure $p_{model}(G; \Theta)$ is close to $p_{data}(G)$
3. Ensure we can sample from $p_{model}(G)$





Graph generative modeling

Ensure $p_{model}(G; \Theta)$ is close to $p_{data}(G)$

- Apply maximum likelihood

$$\hat{\Theta} = \max_{\Theta} \mathbb{E}_{\mathbf{x} \sim p_{data}} \log p_{model}(\mathbf{x} | \Theta)$$

- Find parameters $\hat{\Theta}$ such that for observed data points $\mathbf{x}_i \sim p_{data}$ the $\sum_i \log p_{model}(\mathbf{x}_i | \hat{\Theta})$ is maximized compared to all other choices of Θ
- In other words, find the model that makes the observed data most likely
 - For the Bayesians – this is the mirror of finding the model that is most likely given the data which would be maximizing $p_{model}(\Theta | \mathbf{x})$



Graph generative modeling

Ensure we can sample from $p_{model}(G)$

- One approach
 - Sample from vectors from some chosen distribution
$$\mathbf{z}_i \sim \mathcal{N}(0,1)$$
 - Transform the sample via f_θ
$$\mathbf{x}_i = f_\theta(\mathbf{z}_i)$$
- How do we design f_θ ? Neural networks (of course!).



Autoregressive models

- A single, $p_{model}(\mathbf{x}|\Theta)$, is used for density estimations and sampling
 - Other methods such as variational autoencoders (VAEs) and generative adversarial networks (GANs) use two (2) models, one for each role
- Autoregressive approach considers joint probability distribution over iterations, t , expressed as the product of conditional probabilities (chain rule of probability)

$$p_{model}(\mathbf{x}|\Theta) = \prod_{t=1}^n p_{model}(x_t|x_1, \dots, x_{t-1}; \Theta)$$

where each x_t is an event in $\mathbf{x}$. For example, x_t is the t^{th} word in a sentence

- In graph generation, x_t is the addition of a node or edge

How can we implement this?

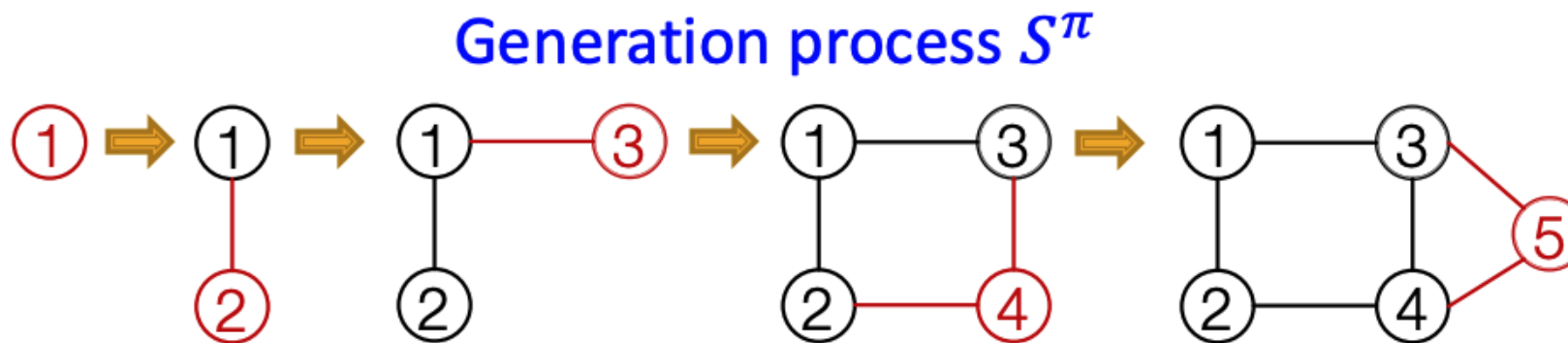
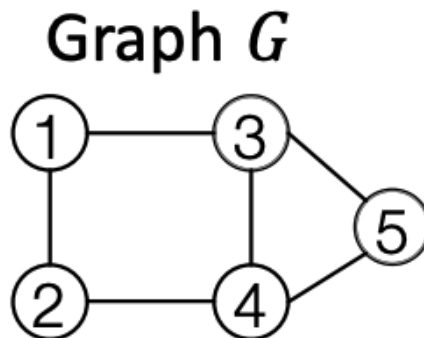


Graphs as sequences



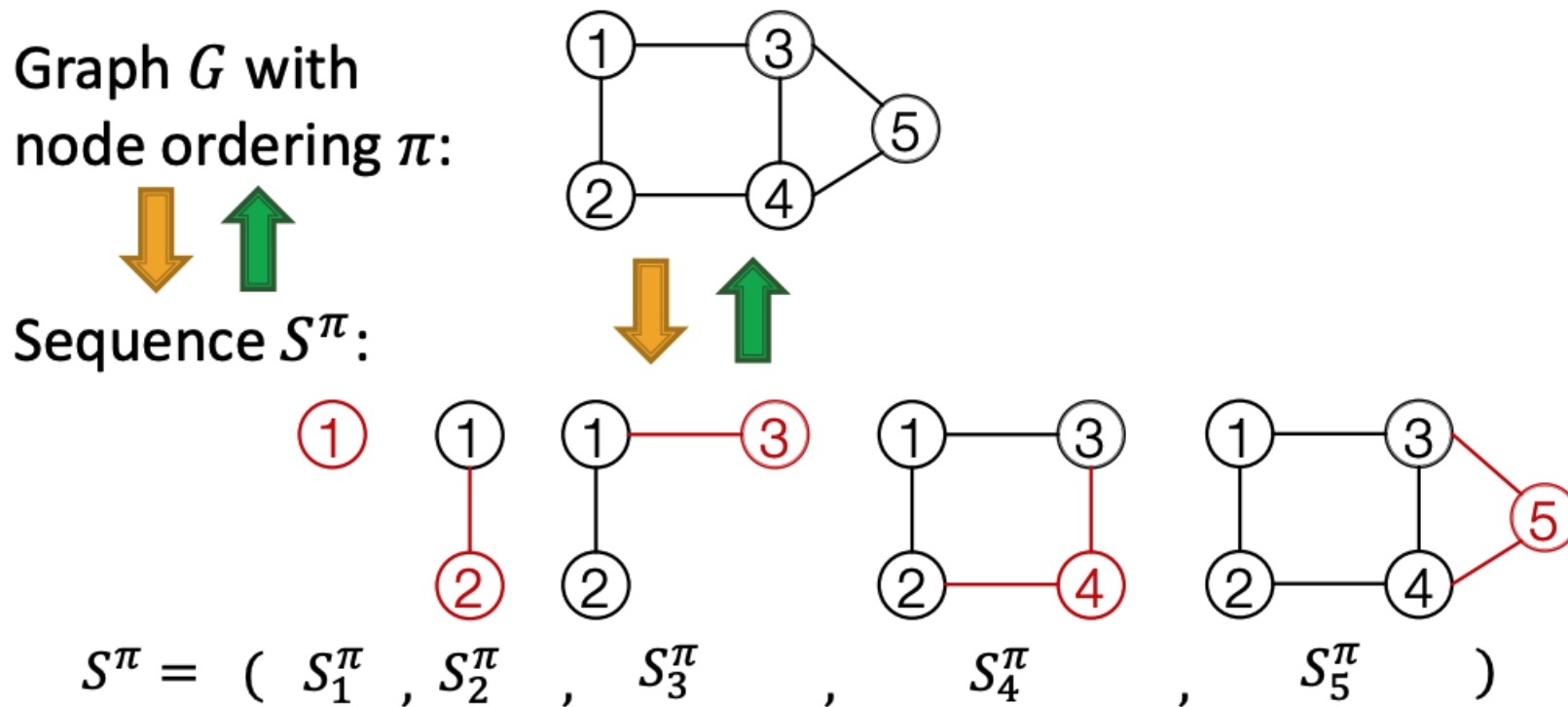
Graph generation as a sequence

Generate graphs via sequentially adding nodes and edges



Graph generation as a sequence

Graph G with node ordering π can be **uniquely** mapped into a sequence of node and edge additions, S^π



The sequence, S^π , has **two levels** (i.e., it is a sequence of sequences)

- Node level:** At each step, add a new node

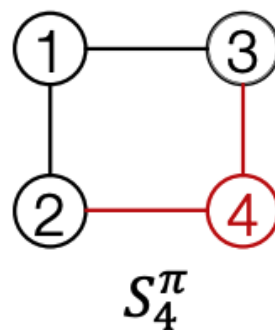


Graph generation as a sequence

The sequence, S^π , has **two levels** (i.e., it is a sequence of sequences)

- Node level: add a new node
- **Each node-level step** creates an edge-level sequence

Edge level: Sequence of edge additions for the new node



Add node 4. Create sequence indicating addition of edges between node 4 and existing edges

$$S_4^\pi = (\underbrace{S_{4,1}^\pi}_{\text{"Not connect 4, 1"}}, \underbrace{S_{4,2}^\pi}_{\text{"Connect 4, 2"}}, \underbrace{S_{4,3}^\pi}_{\text{"Connect 4, 3"}})$$

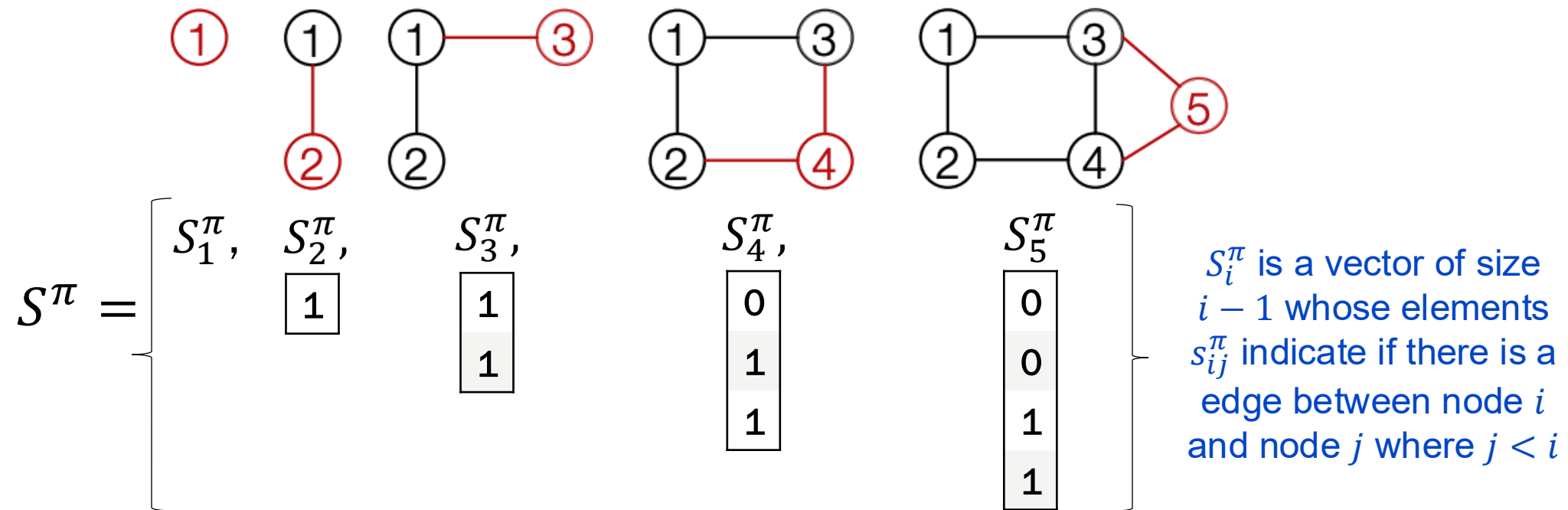
0 1 1

Graph generation as a sequence

The sequence, S^π , has **two levels** (i.e., it is a sequence of sequences)

- Node level: add a new node
- **Each node-level step** creates an edge-level sequence

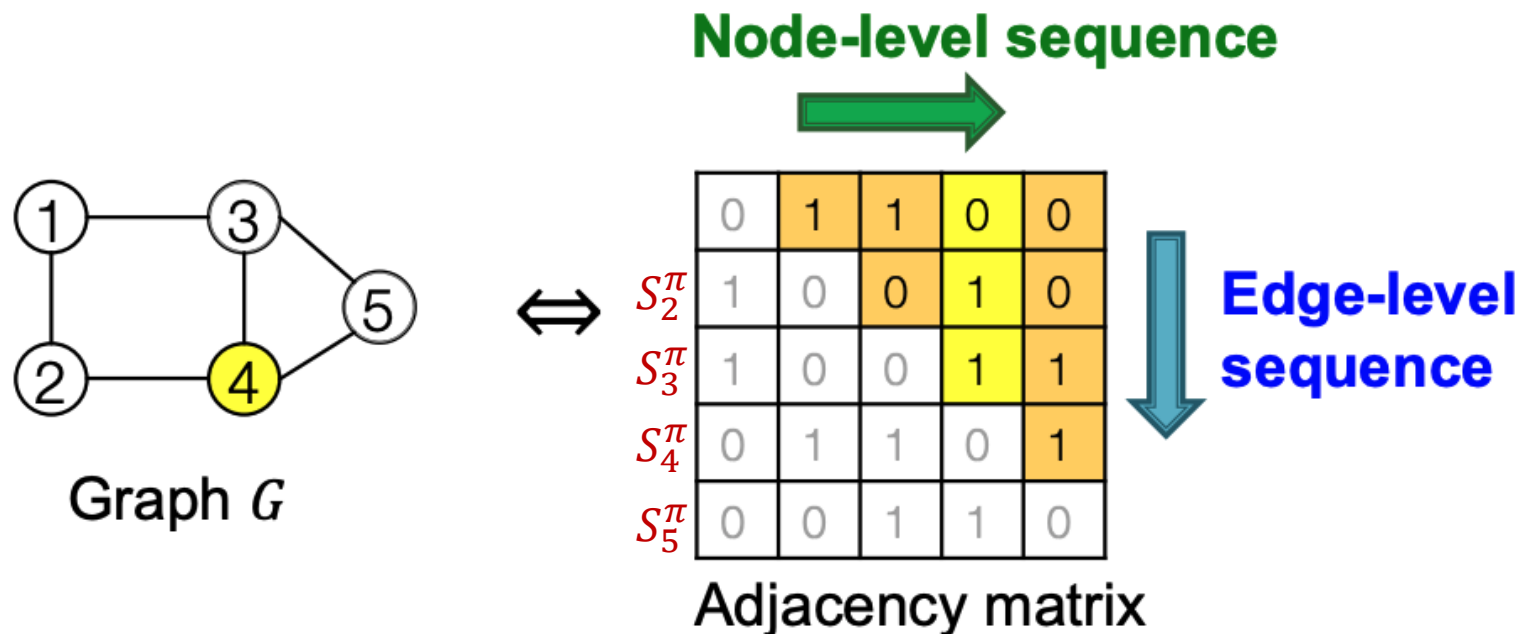
Edge level: Sequence of edge additions for the new node



Graph generation as a sequence

- A graph plus a node ordering $\rightarrow$ a sequence of sequences
- Node ordering is randomly selected (we'll revisit this)
- Why does this matter?

Transformed graph generation into sequence generation





Graph generation as a sequence

- Transformed graph generation into sequence generation
- Can model this as two processes
 1. Generate a state for a new node (node-level sequence)
 2. Generate edges for the new node based on its state
- Can use recurrent neural networks (RNNs) to model these processes
- Could also use Transformers



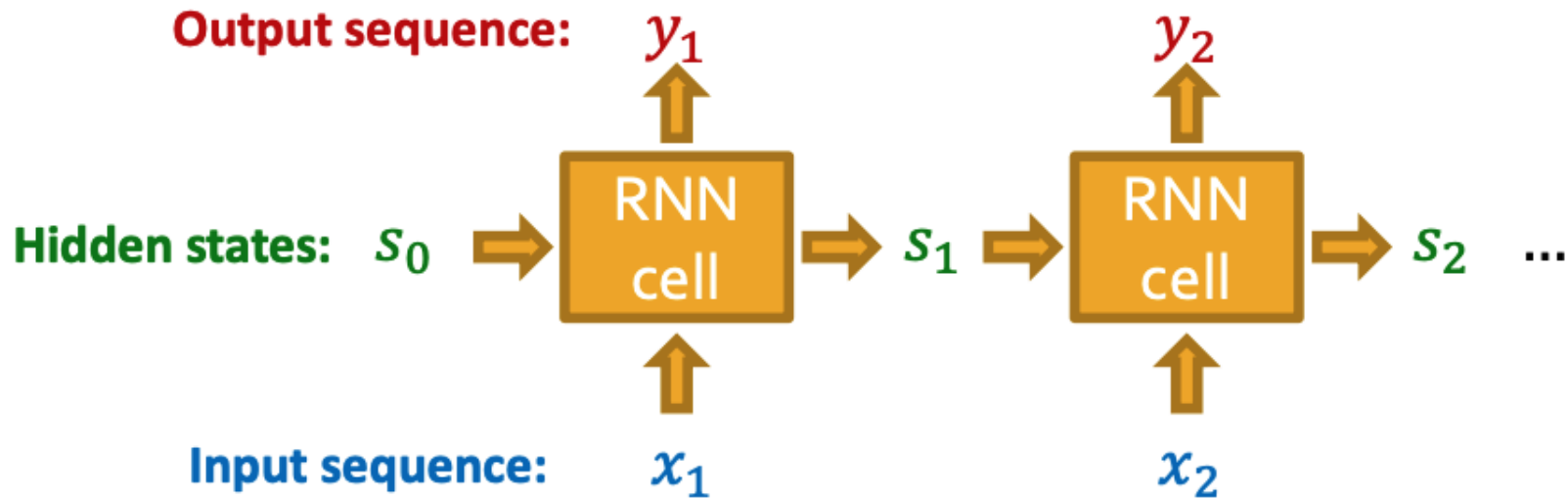
GraphRNN





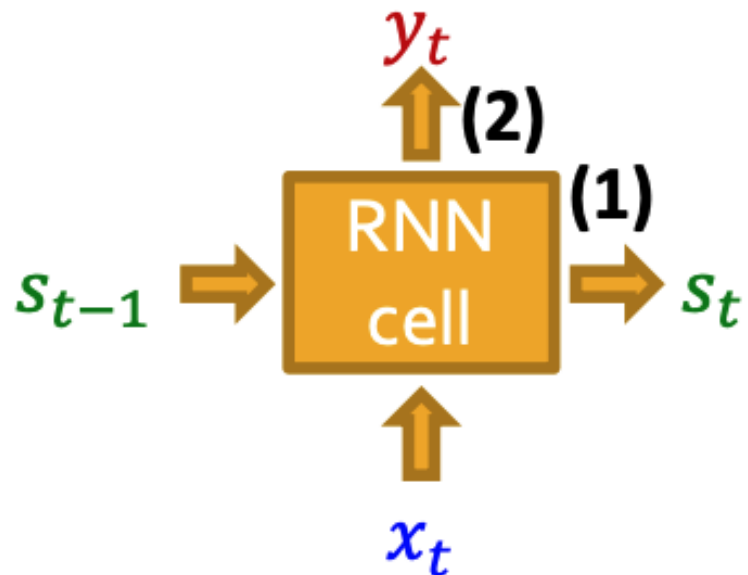
Recurrent neural network (RNN) review

- RNNs are designed for sequential data
 - Sequentially process input elements to update RNN hidden state
 - The hidden state summarizes previous inputs of the sequence
 - The output and hidden state update are created via RNN cells



Recurrent neural network (RNN) review

- s_t : State of RNN after step t
- x_t : Input to RNN at step t
- y_t : Output of RNN at step t



Basic (vanilla) RNN Cell

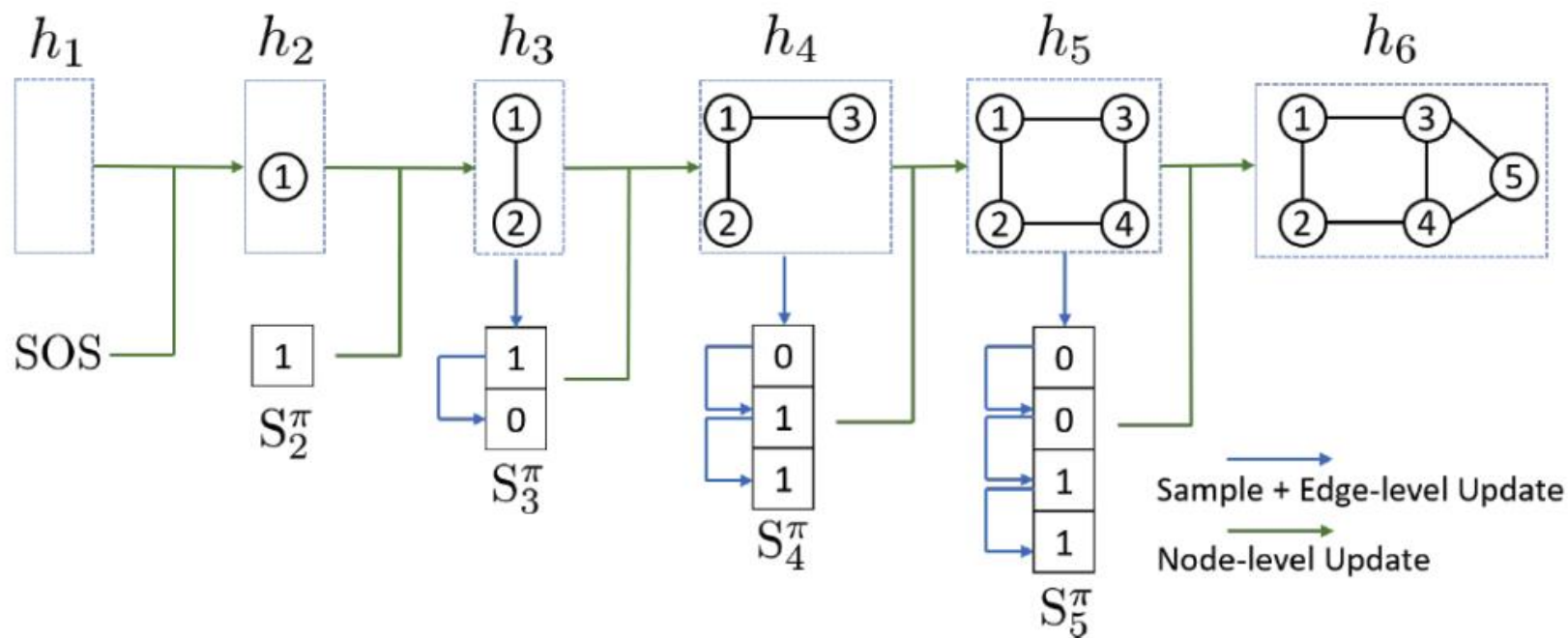
1. Hidden state update
$$s_t = \sigma(Wx_t + Us_{t-1})$$
2. Output

$$y_t = Vs_t$$

- More expressive RNN cells include GRUs, LSTM, xLSTM

GraphRNN: Two levels of RNN

- GraphRNN has a **node level RNN** and an **edge level RNN**
- Relationship between the two RNNs:
 - Node level RNN** generates the initial state for the **edge level RNN**
 - Edge level RNN** sequentially predicts if the new node is connected to each of the previous nodes





GraphRNN for graph sequence generation

Issues to address:

1. How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
2. How to generate the sequence with stochastic sampling?
3. How to conduct training?
4. Creating a new graph?



GraphRNN for graph sequence generation

Issues to address:

1. How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
 - Initialize hidden node level state as a zero vector and input with Start of Sequence (SOS) vector (usually zeros or ones)
 - Provide output of node level as initial state of edge level RNN and input with SOS (this will repeat for every new node)



GraphRNN for graph sequence generation

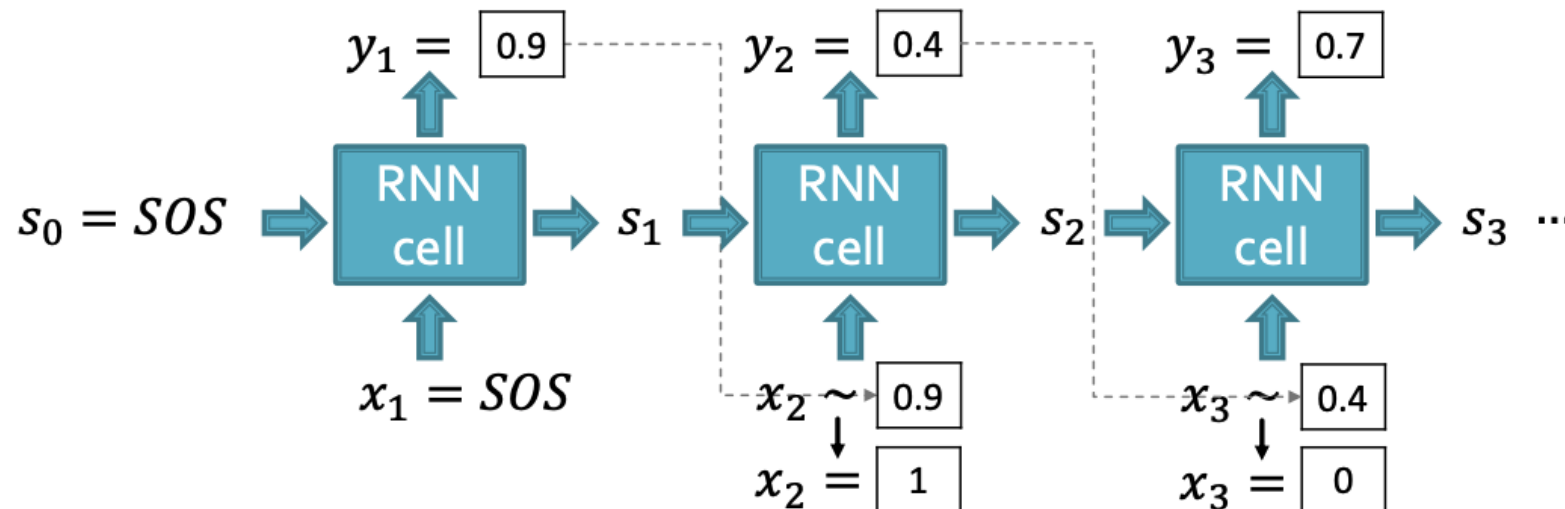
Issues to address:

1. ✓ How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
2. How to generate the sequence with stochastic sampling?
3. How to conduct training?
4. Creating a new graph?

Edge sequence generation

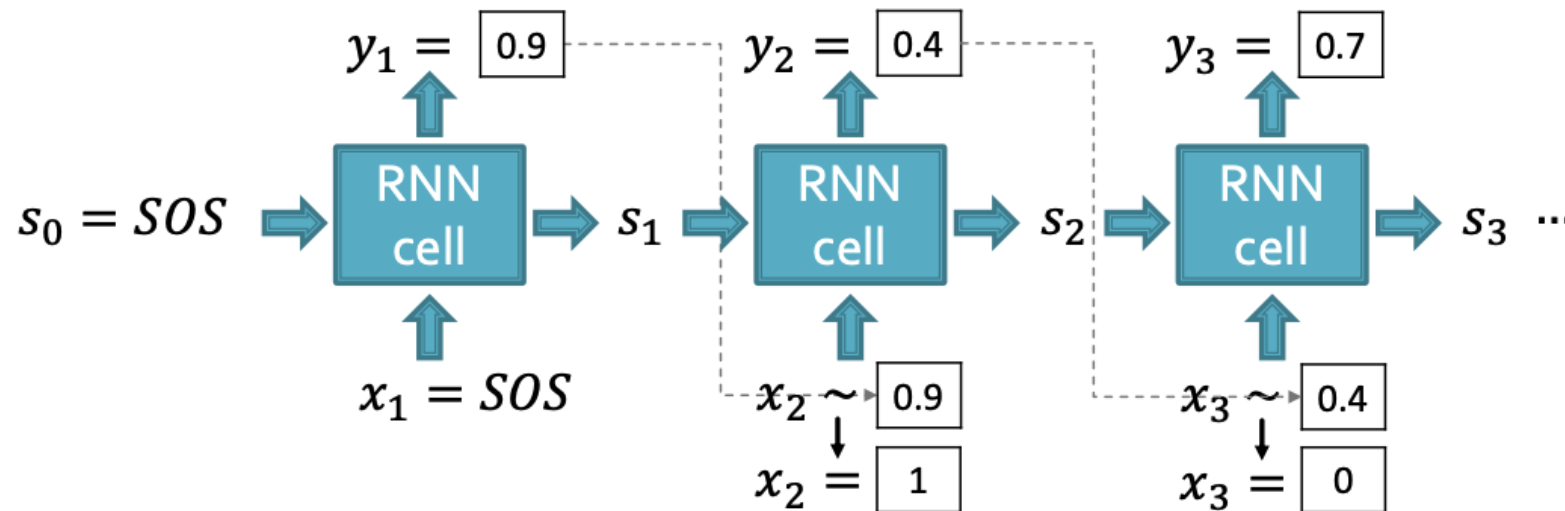
Consider the edge level RNN for now

- Our goal is to model $\prod_{t=1}^n p_{model}(x_t | x_1, \dots, x_{t-1}; \Theta)$
- Let x_t be the event that an edge is added between the new node and an existing node



Edge sequence generation

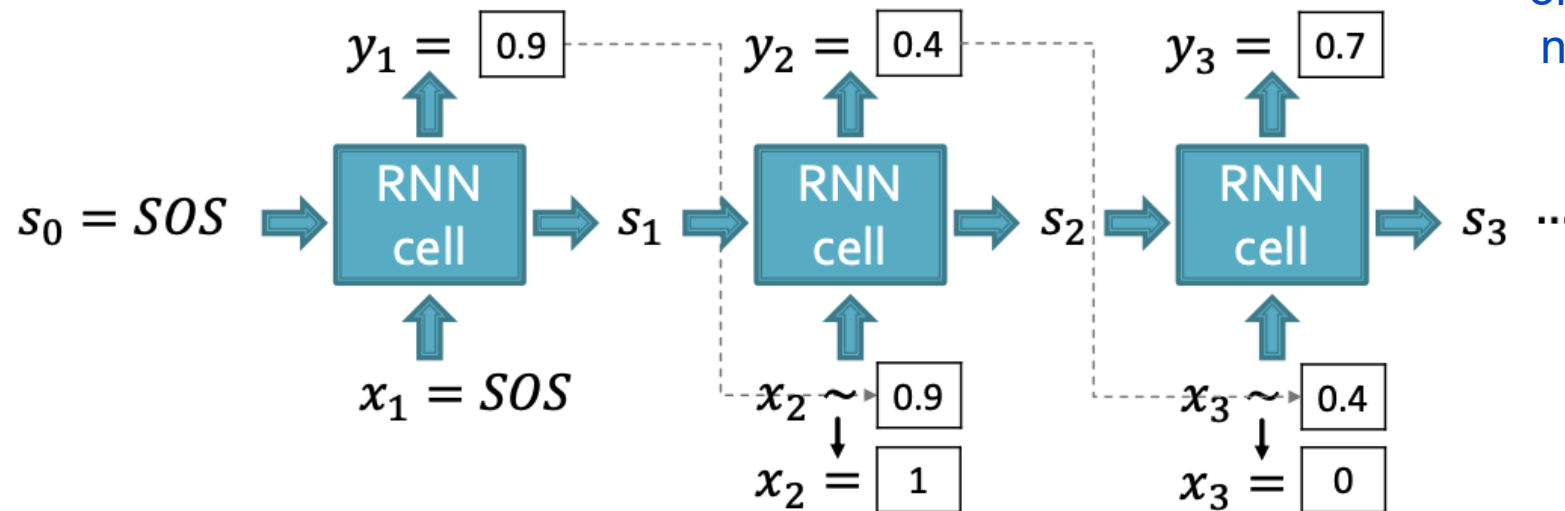
- Let x_t be the event that an edge is added between the new node and an existing node
- We want the edge level RNN output $y_t = p_{model}(x_{t+1} | x_1, \dots, x_t; \Theta)$
- We can then stochastically add edges by sampling x_{t+1} from y_t



Edge sequence generation

Assume the edge level RNN is already trained

- Interpret y_t as the probability of success in a Bernoulli distribution
- $y_t = \boxed{p}$ below indicates value 1 (add edge) has probability p , value 0 (no edge) has probability $1 - p$
 - Sample a random number, r , from a uniform distribution on $[0,1]$
 - If $r \leq y_t$ add an edge, otherwise no edge
 - Repeat for all nodes currently in the graph



Stochastic process generates a sample S_i^π of edges between new node i and nodes j in the current graph.



GraphRNN for graph sequence generation

Issues to address:

1. ✓ How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
2. ✓ How to generate the sequence with stochastic sampling?
3. How to conduct training?
4. Creating a new graph?



Edge level RNN training

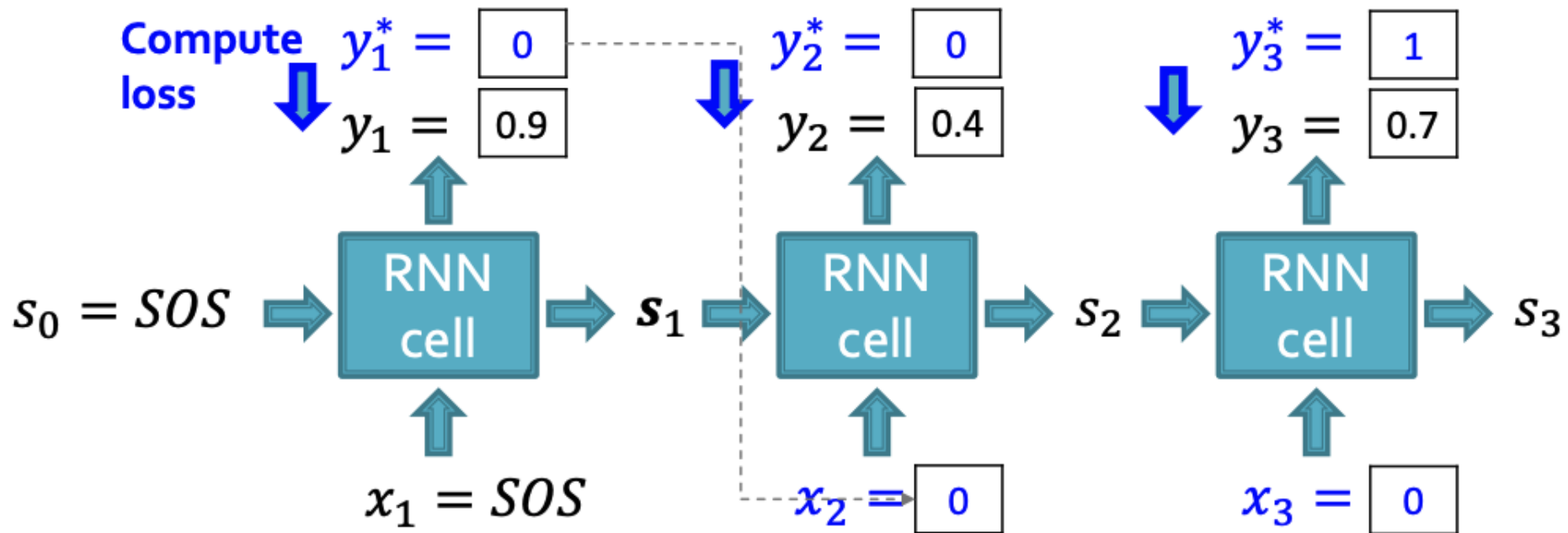
Training the model

- Observe a training sample with a sequence of edges $[0, 0, 1, \dots]$
- Apply teaching forcing principle – at each step replace the predicted output with the actual value (training only)
 - Teaching forcing principle leads to faster training and improved convergence (usually) for RNNs

Edge level RNN training

Training the model

- Observe a training sample with a sequence of edges $[0, 0, 1, \dots]$
- Apply teaching forcing principle – at each step replace the predicted output with the actual value (training only)





Edge level RNN training

- Minimize binary cross entropy loss

$$\mathcal{L} = -[y_1^* \log(y_1) + (1 - y_1^*) \log(1 - y_1)]$$

- For **every pair of nodes** in a training graph:
 - If $y_1^* = 1$ (an edge is present), minimize $-\log(y_1)$ so y_1 increases
 - If $y_1^* = 0$ (no edge is present), minimize $-\log(1 - y_1)$ so y_1 decreases
 - y_1 is fitting to the training samples – learned edge probability distribution should match data distribution
 - Remember y_1 is computed by the combined RNNs, so the loss gradients will adjust RNN parameters



GraphRNN generation plan

Initialization

- $S_1^\pi = SOS$ (node 1 doesn't have previous nodes to connect)
- $h_1 = empty$ (Node RNN initial state)
- $i = 1$

Generation

1. $i = i + 1$
2. **Add a new node:** Run the Node RNN to generate output x_i and updated state h_i
3. **Generate S_i^π :** Run the Edge RNN to predict if the next new node will connect to existing nodes (including the new node from step 2)
 - a. If $S_i^\pi = EOS \rightarrow Stop$
 - b. Else: go to step 1



GraphRNN generation plan

Generation

1. $i = i + 1$
2. **Add a new node:** Run the Node RNN to generate output x_i and updated state h_i
3. **Generate S_i^π :** Run the Edge RNN to predict if the next new node will connect to existing nodes (including the new node from step 2)
 - a. If $S_i^\pi = EOS \rightarrow \text{Stop}$
 - b. Else: go to step 1

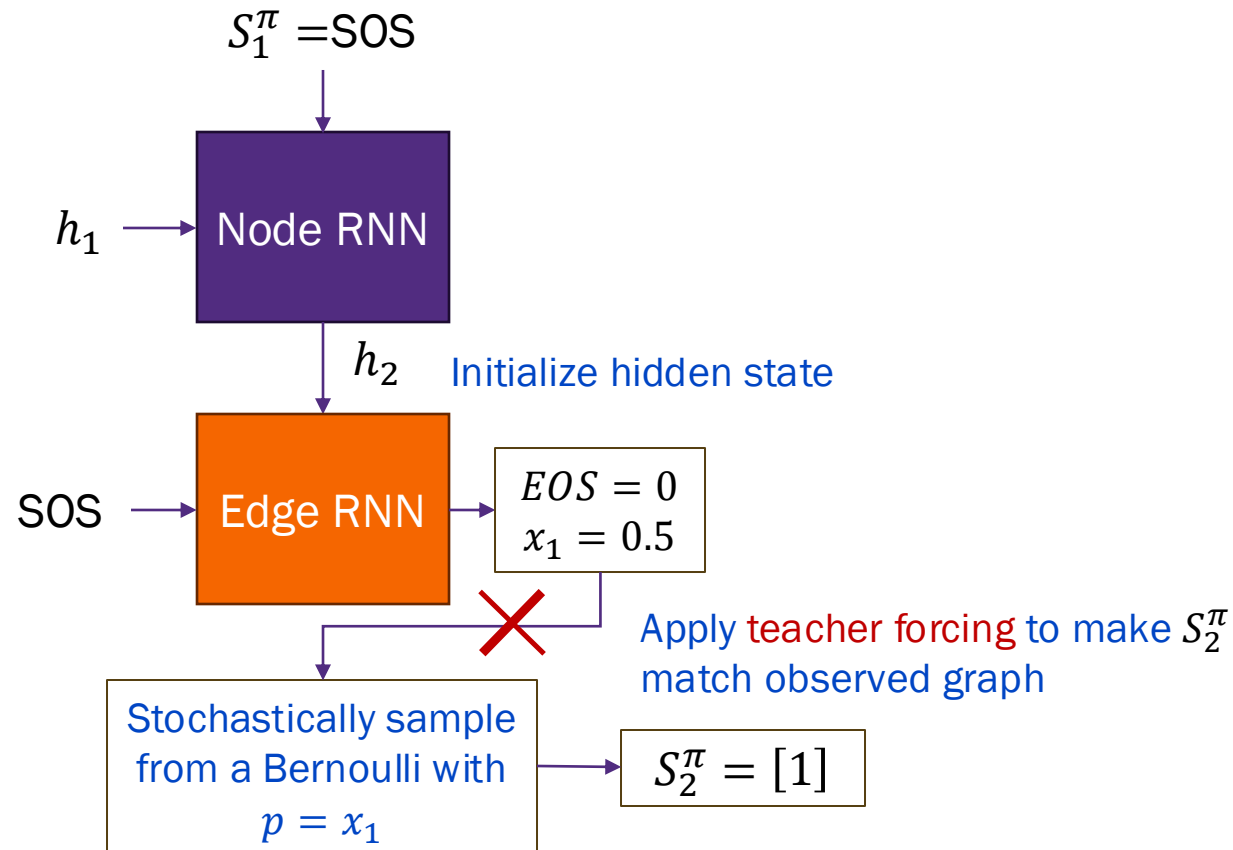
Note: S_i^π are the predicted edge values for the next new node. For example, for $i = 2$, after step 2, there is one (1) node in the graph (created at step 1). $S_2^\pi = [s]$ where $s \in [0,1]$ indicates whether node 2, if created, will connect to node 1.

Why predict ahead? If the EOS is generated in this iteration, we want the node state vector, h_i , to represent the graph with its current nodes.

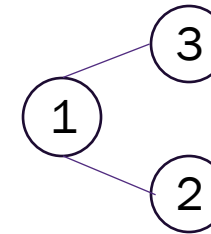


GraphRNN training

- Initial state: $S_1^\pi = SOS$, $h_1 = \text{empty}$



Training sample



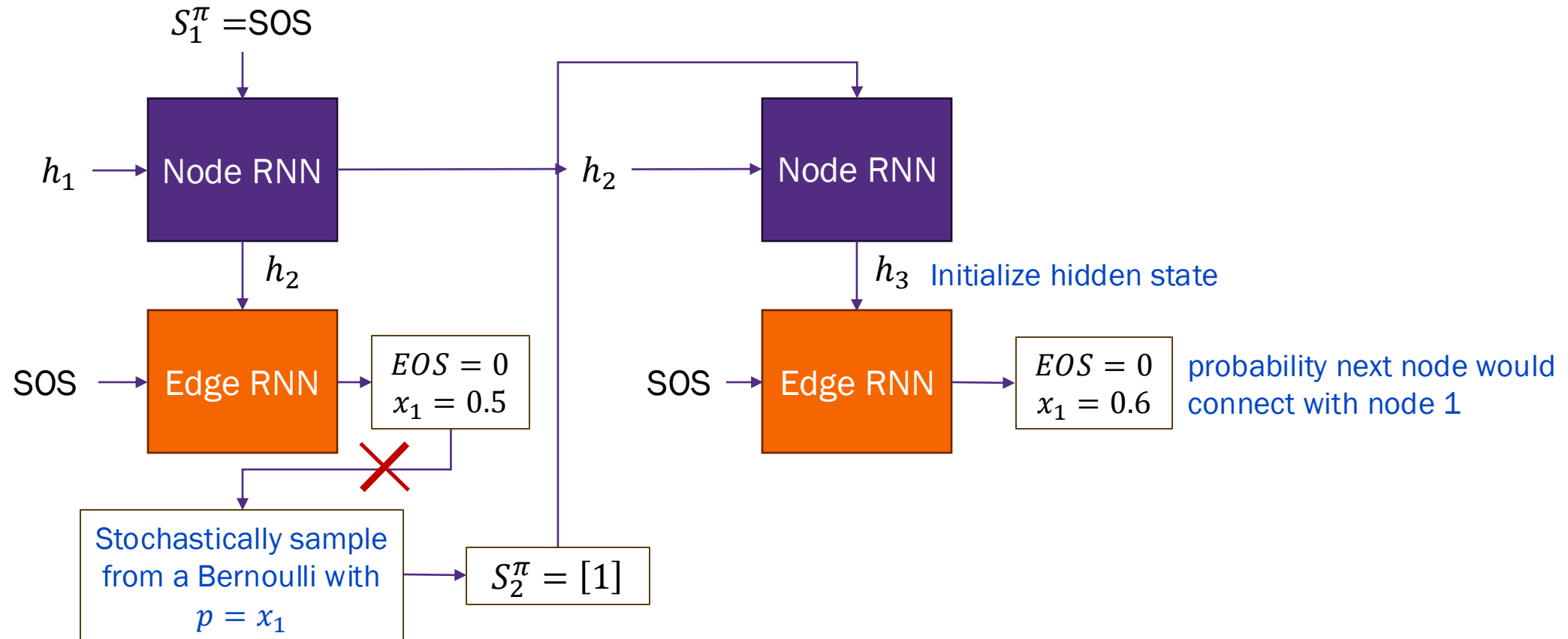
Generated graph



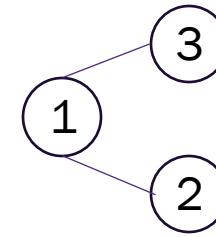


GraphRNN training

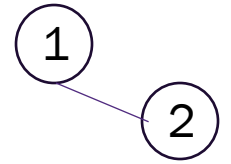
- Current state: $S_2^\pi = [1]$, $h_2 \rightarrow 1$ node



Training sample

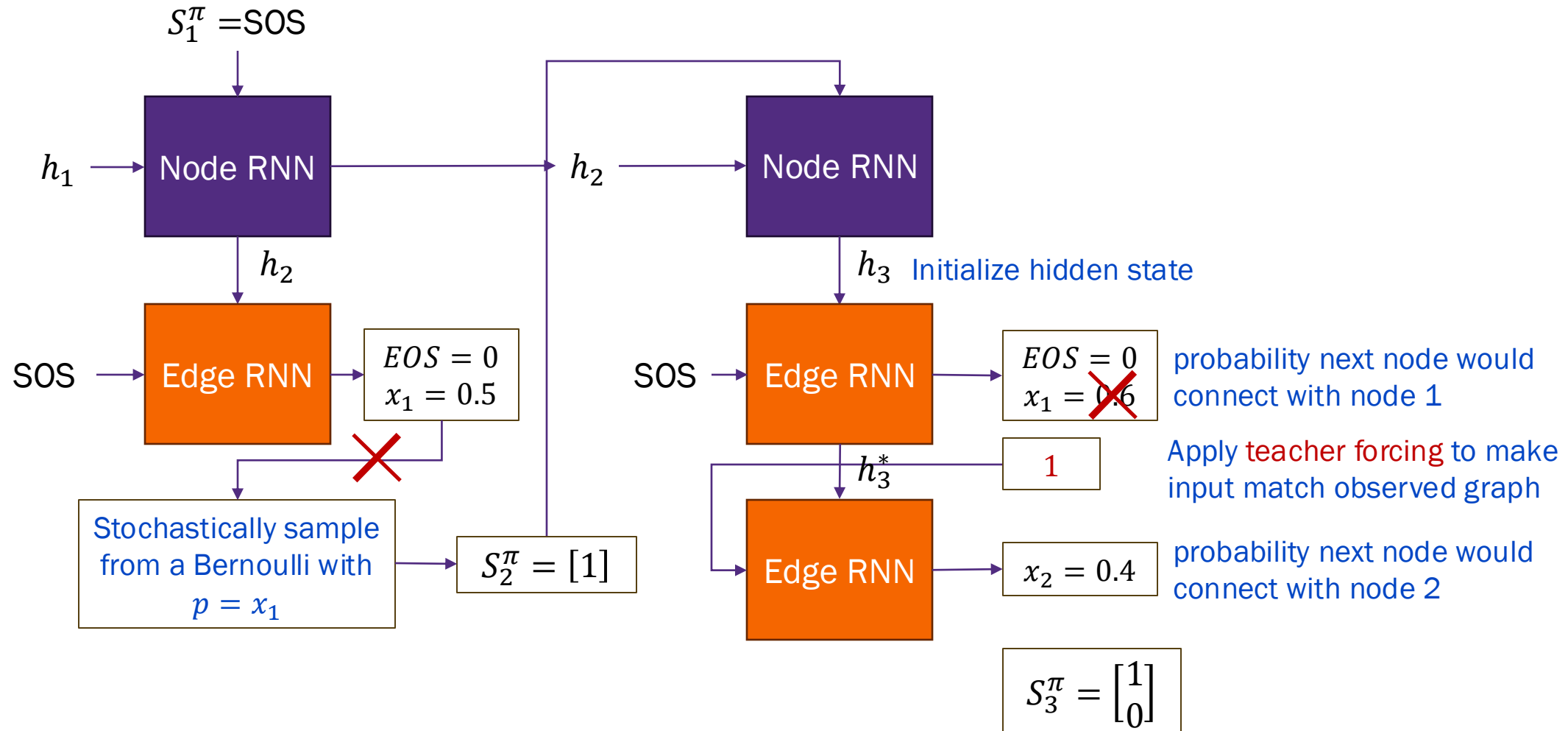


Generated graph



GraphRNN training

- Current state: $S_2^\pi = [1]$, $h_2 \rightarrow 1$ node

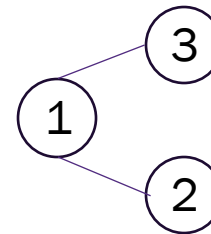




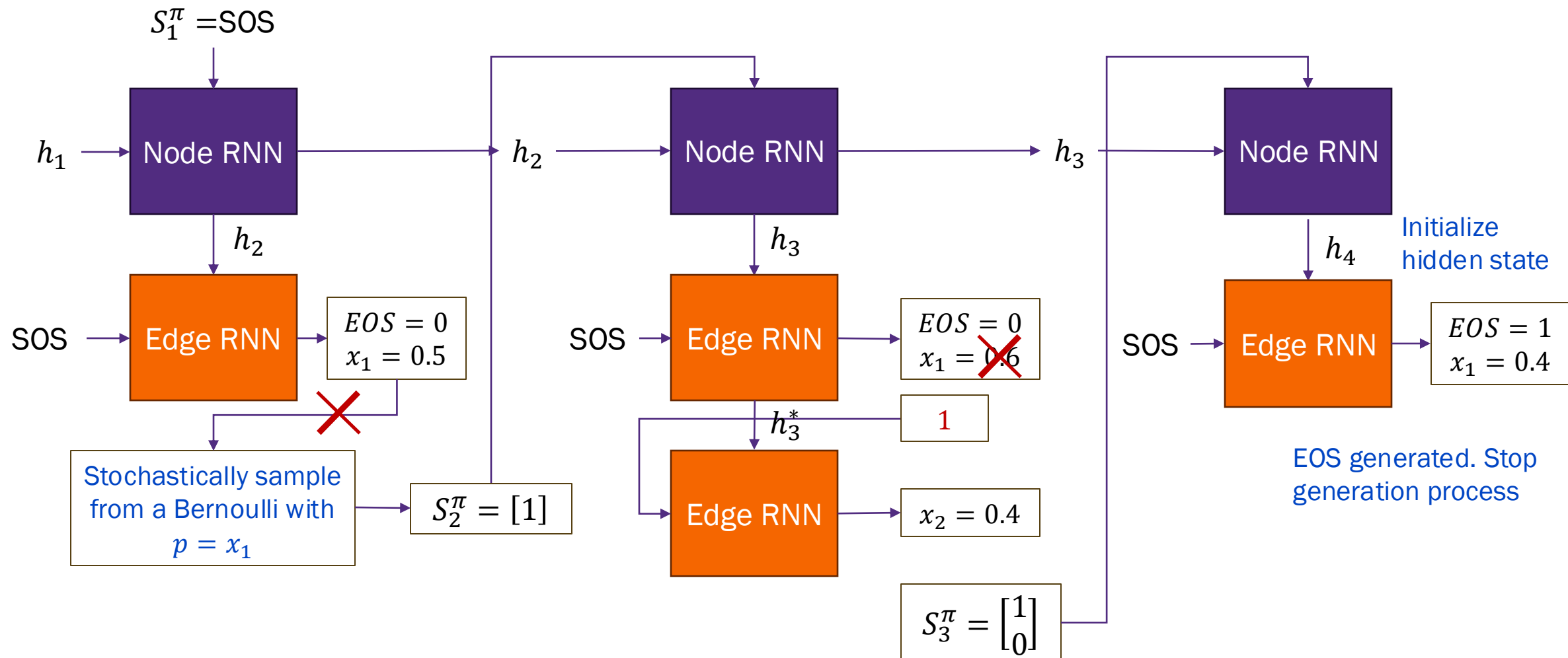
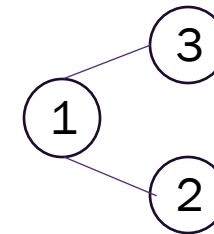
GraphRNN training

- Current state: $S_3^\pi = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $h_3 \rightarrow 2$ nodes

Training sample



Generated graph

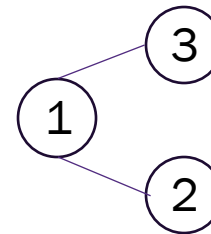




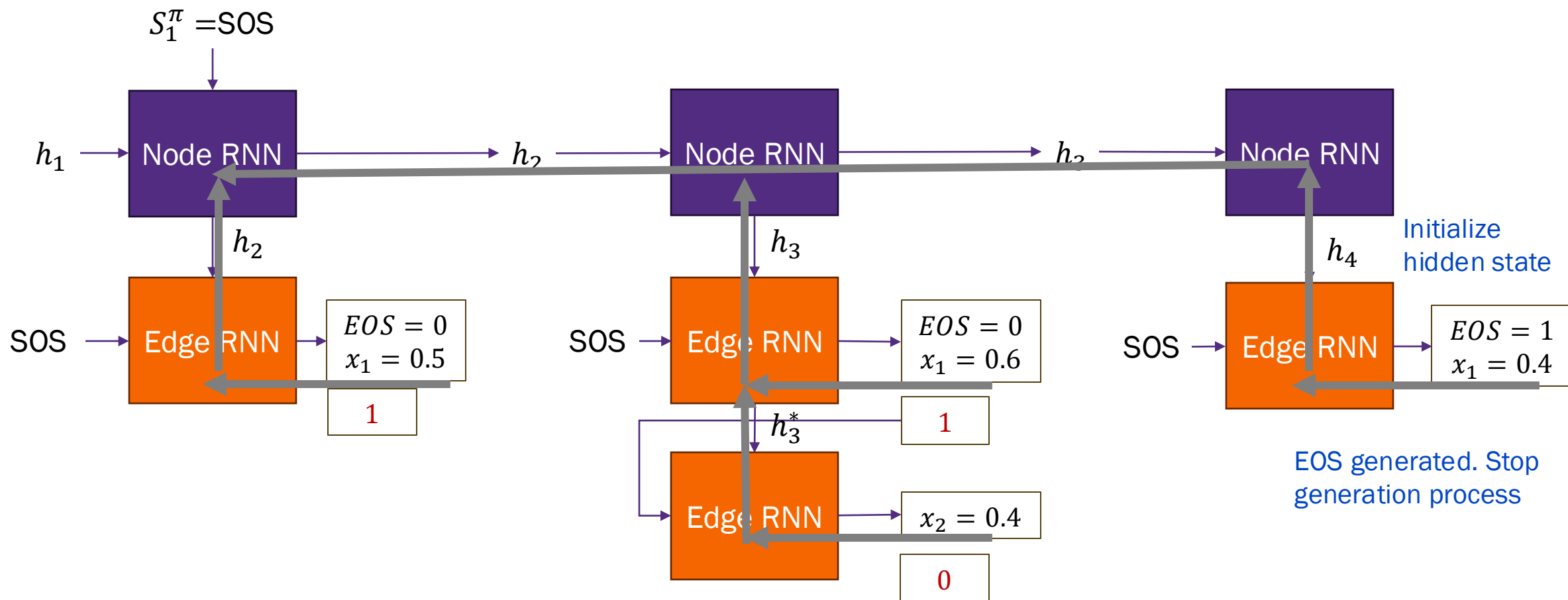
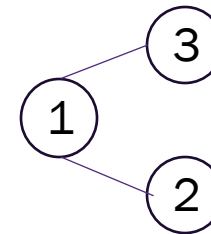
GraphRNN training

- Backprop through time. Loss gradients accumulated across iterations.
- Update parameters

Training sample



Generated graph

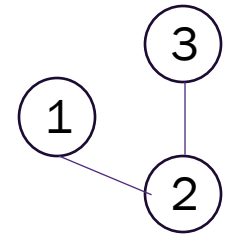




GraphRNN for graph sequence generation

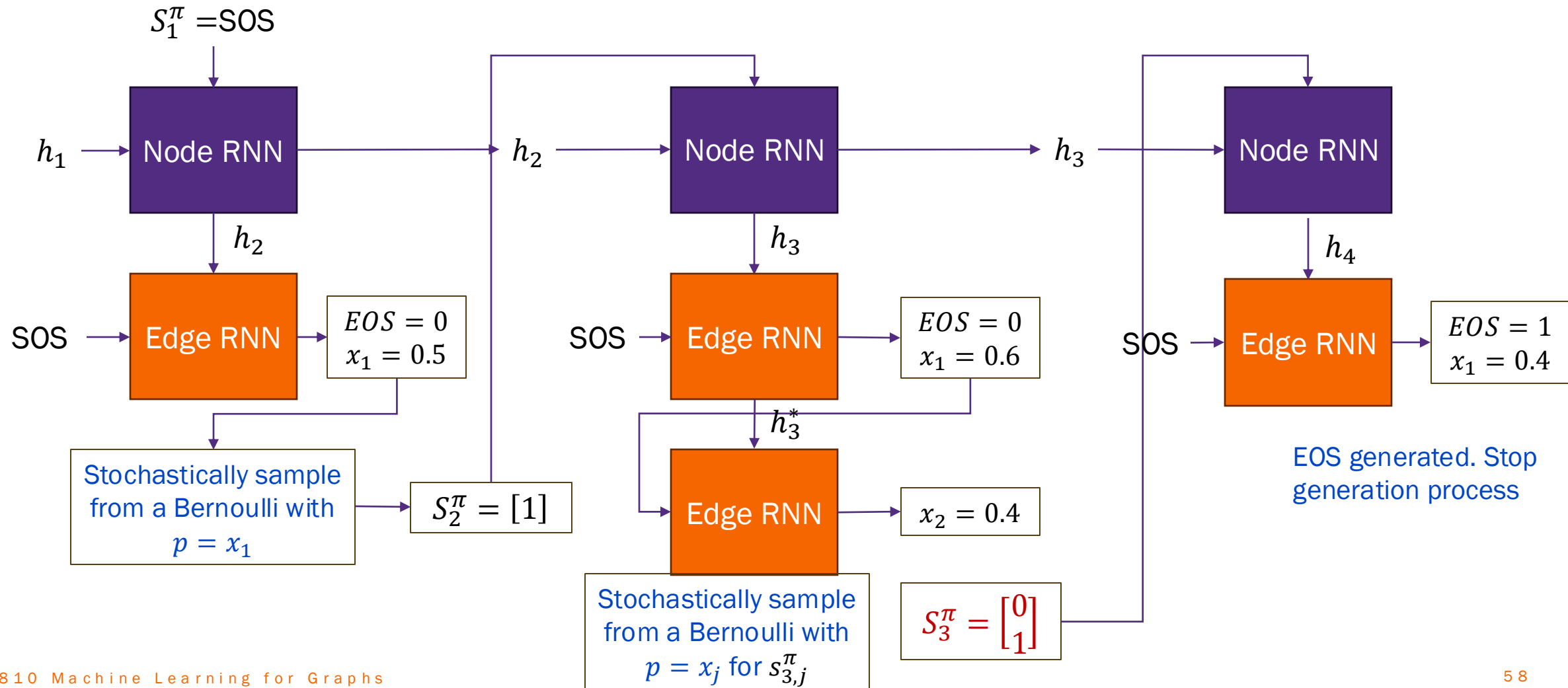
Issues to address:

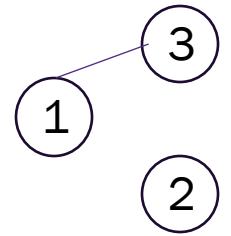
1. ✓ How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
2. ✓ How to generate the sequence with stochastic sampling?
3. ✓ How to conduct training?
4. Creating a new graph?



GraphRNN graph generation

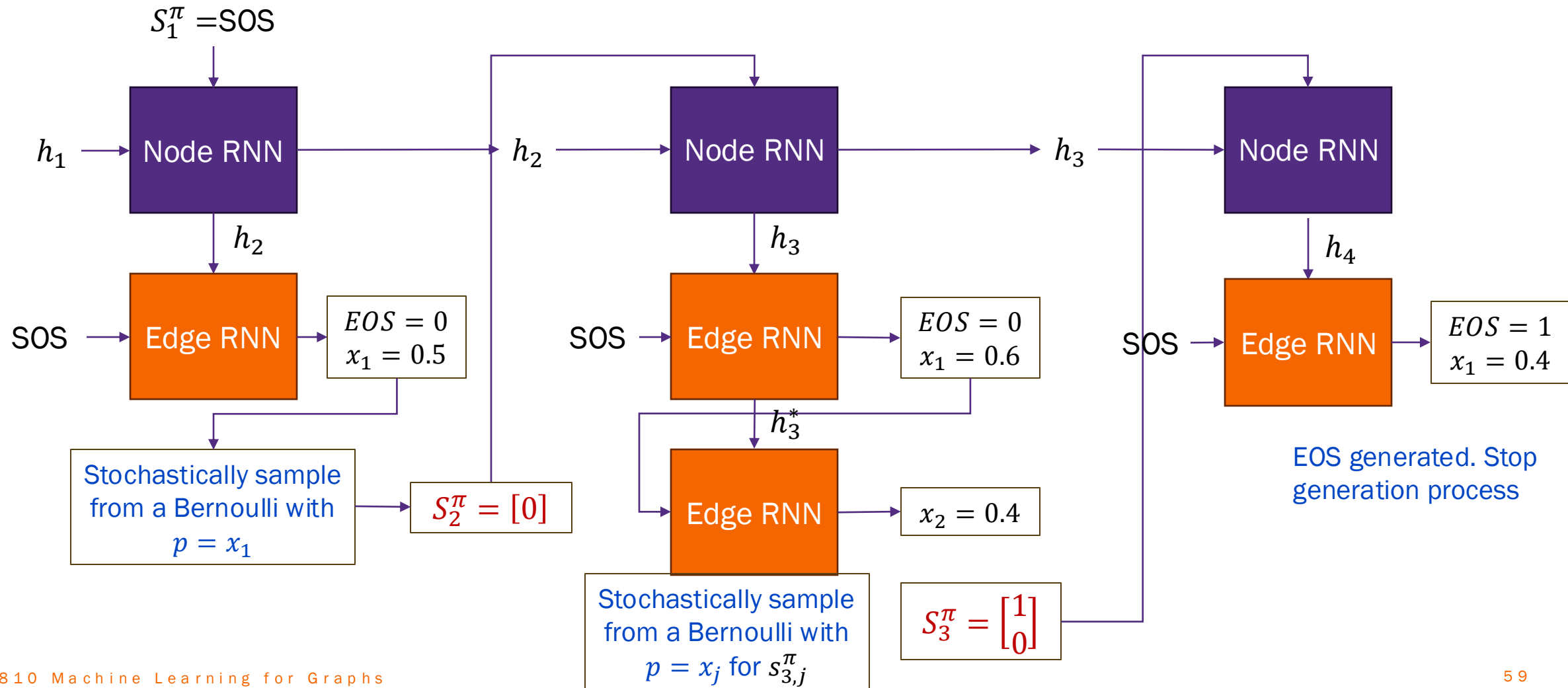
- Current state: $S_3^\pi = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $h_3 \rightarrow 2$ nodes





GraphRNN graph generation

- Current state: $S_3^\pi = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $h_3 \rightarrow 2$ nodes





GraphRNN for graph sequence generation

Issues to address:

1. ✓ How to initialize the hidden state and input sequence for the node level RNN? We don't have any nodes yet.
2. ✓ How to generate the sequence with stochastic sampling?
3. ✓ How to conduct training?
4. ✓ Creating a new graph?

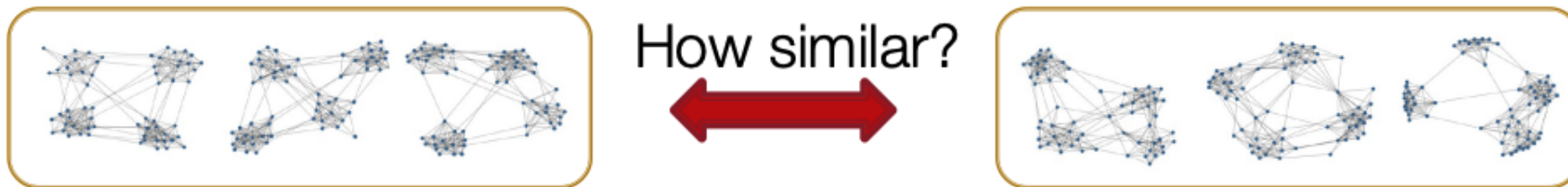


Evaluation



Evaluating generated graphs

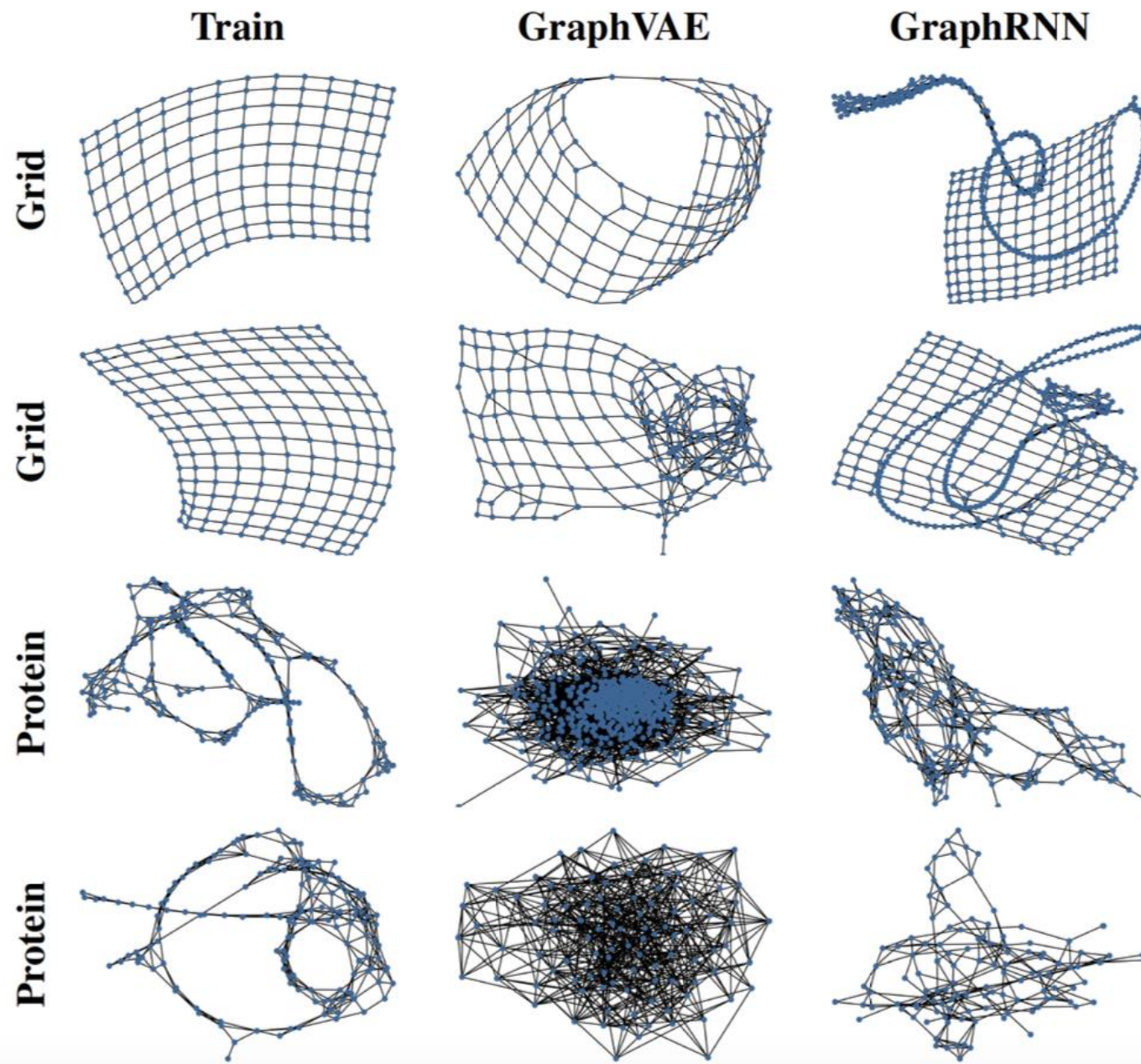
How to compare two sets of graphs



- Approaches
 1. Visual inspection (qualitative)
 2. Statistical metrics



Visual similarity





Statistical measures

- Compare univariate statistics of the generated graphs against a test set
- Assume we have a set of graph statistics $S = \{s_1, \dots, s_n\}$ where each statistic $s_{i,G}: \mathbb{R} \rightarrow [0,1]$ defines a univariate distribution over $\mathbb{R}$ for a given graph G .
- Compute the distance between the distributions for the test graph and the generated graph using a distributional measure such total variation distance (TVD)

$$d(s_{i,G_{test}}, s_{i,G_{gen}}) = \sup_{x \in \mathbb{R}} |s_{i,G_{test}}(x) - s_{i,G_{gen}}(x)|$$

- $\text{TVD} = 0 \rightarrow$ perfect match
- $\text{TVD} = 1 \rightarrow$ no match



Statistical measures example

- Calculate the node degree for all nodes in G
- Form the degree distribution.
 - For each observed degree, d , find the number of nodes, n_d , with that degree.
 - The statistic $s_{i,G}(d) = n_d / |G|$ where $|G|$ is the number of nodes in G
- Compute the total variational distance (TVD)

$$d(s_{i,G_{test}}, s_{i,G_{gen}}) = \sup_{x \in \mathbb{R}} |s_{i,G_{test}}(x) - s_{i,G_{gen}}(x)|$$

For each observed degree, calculate the absolute difference between $s_{i,G}(d) = n_d / |G|$ for the test graph and generated graph and take the maximum.

- Repeat across multiple graphs to get a mean

